

Nonuniform Temporally Stochastic Cellular Automata as Model of Contagious Disease Spread

Subrata Paul

*Department of Information Technology
Indian Institute of Engineering Science and Technology
Shibpur, Howrah, West Bengal — 711103, India*

Kamalika Bhattacharjee*

*Department of Computer Science and Engineering
National Institute of Technology
Tiruchirappalli, Tamilnadu — 620015, India
{psubrata.it, kamalika.it*}@gmail.com*

**Corresponding author*

In this paper, we introduce a new cellular automaton (CA) variant called the nonuniform temporally stochastic CA (NTSCA), where an NTSCA contains a sequence of temporally stochastic cellular automata (TSCAs). Different TSCAs are used for different time frames during the evolution of the model to simulate the propagation of contagious viruses. We also propose the idea of a distant neighbor to depict the spread of viruses that occurs when asymptotic carriers are transported large distances. The essence of this spread is embodied by a specific set of features. These features are included in the NTSCA by utilizing information propagation, information cooking and partial blocking of information. Based on these parameters, the required rules are identified and used in the applied TSCAs. The dynamical changes of TSCAs during evolution make the system chaotic and enable imitating the propagation of diseases. We successfully imitate the spread of two such contagious diseases borne by viruses, COVID-19 (in India) and SARS 2003 (in China) and compare the results with real-time data.

Keywords: nonuniform temporally stochastic cellular automata; NTSCAs; distant neighbor; information propagation; information cooking; partial blocking; epidemic; COVID-19; SARS 2003

1. Introduction

A coronavirus is associated with the source of the viral respiratory disease known as severe acute respiratory syndrome (SARS). Coronaviruses are airborne viruses that are similar to cold and flu: they are able to spread by small droplets of saliva. In the twenty-first century, SARS was the first contagious deadly virus that emerged over several

countries. SARS showed a potential to spread along the routes of international air travel [1] and may spread indirectly through surfaces touched by someone infected with the virus. On November 16, 2002, the first case of SARS was identified in Guangzhou, China. Later, it spread to 26 countries in Europe, America and Asia, making it a global epidemic [2]. This deadly virus, which is highly contagious and can produce certain super-spreaders, is not the only occurrence of such viruses in the modern period; H1N1 (2009, 2010), MERS (2012), Zika (2015, 2016), Ebola (2014 and 2018), COVID-19 (2019), monkeypox (2022) and other contemporary diseases have caused devastating effects in various regions of the world [3, 4].

The most dangerous disease in recent times has been the COVID-19 pandemic that had an awful effect from 2019–2022, straining the healthcare systems of various countries. Although it has a low mortality rate, the extremely high infection rate of COVID-19 made it a devastating disease that stopped almost all everyday activities worldwide and killed more than six million people.

These scenarios highlight the need for developing computational and mathematical models for evaluating the rapidity with which a virus spreads throughout the population, for at least two reasons. First, it is essential to apply these data-driven results in order to predict the spread of different kinds of viruses. If this situation recurs in the future, it will be easier to predict what is going to happen, such as whether the system will become stable, based on the present course of action. Second, and probably more importantly, these models may be used to understand the features of a pandemic, such that it will be easier to recognize potential pandemic-causing viruses before they become widespread and check for the likelihood of new pandemics in the future. This will help us to prepare ourselves for the avoidance and management of such situations.

When an infectious virus spreads and mutates over time, it may become deadly. This suggests that the behavior of a virus evolves over time. Likewise, any model effective in simulating the spread of such a virus has to be capable of changing its behavior dynamically. For this reason, cellular automata (CAs) have long been an effective method for modeling the development of epidemics, including the spread of deadly diseases, such as Chikungunya, influenza and COVID-19 [5–10]. CAs can contain many of the features of a contagious virus that causes pandemic/epidemic-like situations (Section 2). Hence, we consider CAs as the backbone of our model. In this model, we include viral mutation as well as preventative measures that individuals need to take in order to prevent the spread of deadly contagious viruses.

In order to effectively model the spread of any such disease, we propose a novel kind of cellular automaton (CA) called the nonuniform

temporally stochastic CA (NTSCA). We previously studied temporally stochastic CAs (TSCAs), where two rules f and g were used temporarily with some probability τ [11]. Here, a sequence of these TSCAs is employed in the proposed NTSCA. The model description specifies the time frame during which the TSCAs are in effect. A preliminary version of this work using NTSCA is discussed in [12]. In this paper, the spread of infection and recovery are addressed by rules f and g , respectively. Each TSCA is therefore applied in accordance with the rates of propagation and recovery. To mimic the actual situation where both infectious preferences and immunity to resist the infection are active, the TSCAs are applied accordingly. The development of useful rules for the NTSCA model is discussed in Section 3.

As case studies, we develop two realistic simulations for two different diseases, COVID-19 in India (Section 4) and SARS in China (Section 5). We incorporate human precautions taken to prevent the spread of these highly contagious diseases into the model, together with recent real-time data. It can be observed that the proposed model can simulate the diseases accurately and detect the waves for the COVID-19 and SARS pandemics. Then we fit the real-time data precisely for both cases to check the efficacy of the model. The research concludes with Section 6, where the possibility of modeling other epidemics is discussed.

2. The Model

The earlier models that were suggested are static and specific to a particular disease. However, viruses behave differently in the real world while spreading. By the laws of nature, viruses mutate over time, hence, the impact of a virus changes with respect to time. This property often makes a virus deadlier. These types of complexities have not been sufficiently captured or stated in other studies. In this paper, we look particularly into the propensity for mutation. The proposed model performs as a generic model to mimic the spreading of any such virus. Additionally, this model incorporates distant neighborhoods to simulate how diseases spread over long distances. We first define the features of a virus that causes an epidemic.

2.1 Features of a Highly Contagious Virus

We can observe the following features of a virus to comprehend its dynamics.

1. *Locality of transmission.* A contagious virus can only spread by direct contact with infected people, animals or other things. As a result, a person can only get infected if his/her immediate neighbors have been infected.

2. *Chain of infection.* Through a chain of infection, a contagious virus travels from one person to another. Hence, people can be a carrier of the disease.
3. *High rate of spread.* Almost everyone who comes into close contact with infected (animate or inanimate) things will get infected. This property makes such viruses much deadlier.
4. *External factors.* If nothing blocks its path, a highly contagious virus keeps spreading its infection throughout the entire world. However, after identifying such viruses (e.g., COVID-19), ways to halt the virus spread through government or individual efforts have been attempted. Some strategies follow:
 - to lock down the region and confine the disease to specific locations to stop the future intake of additional infected carriers;
 - to maintain isolation and social distancing;
 - to implement the WHO-recommended preventative safety measures;
 - to immunize individuals with vaccinations.
5. *Mutation of the virus.* The virus itself changes according to the laws of nature, while producing copies of itself in the host's body. The system may ultimately become stabilized by this mutation.

Therefore, to comprehend the dynamics of a highly contagious virus, we need strategies that include these features in the proposed CA.

■ 2.2 Features Included in the Cellular Automata

In order to develop parallel complex dynamic systems, a global framework is provided by customizing the parameters of a CA. CAs can be an ideal choice for simulating the development of infectious diseases that may spread rapidly and cause an epidemic or pandemic, due to several inherent features. We now compare these features to a virus that spreads rapidly.

1. *Locality of transmission.* Similar to the spread of diseases, each CA updates depending on its neighbors, which adhere to a local rule.
2. *Chain of infection.* In a lattice, the changes of a cell spread across the lattice through its overlapping neighborhood structure, much like how an infectious virus acts.
3. *High rate of spread.* We can model the spread of disease as a piece of *information* that *flows* through a lattice in nearly every direction, rendering the system as *chaotic*. This way, we can simulate the disease produced by a highly contagious virus.
4. *High rate of impact.* A disease spreads throughout the lattice depending on the rate of information flow. Therefore, an uninfected cell may influence its neighbors; yet, due to the infection rate of the neighbors, the

cell becomes infected after a period of time and spreads infection. This implies that the cell has some information cooking rate.

5. *The external factors.* An automaton never requires outside input to continue operating. However, to make a realistic model, outside influences must be considered in some way. We may apply the concept of *partial blocking* of information [13], where the spread of information (infection) is (partially) blocked in certain cells to depict lockdown and social distancing. This indicates that a disease is limited in its ability to propagate. A modest amount of information leakage must be permitted in those cells since the total blocking of human interaction is exceedingly challenging, even under tight lockdown and social distance restrictions.
6. *Mutation of the virus.* The applied CA rules must change dynamically (in accordance with the standards of the proposed CA) to mimic viral mutation. We assess this category of CA rules in the following section.

Therefore, our system must be dynamic to replicate the behavior of a virus and account for rapid dissemination because of high contagion rates. Viruses from one geographic location can infect another due to local and international airplane connections among multiple locations. If we let the cells have some *nonlocal* neighbors, the CA is also able to manage the *nonlocal/distant* virus spread. The nature of a highly contagious virus can be considered as a chaotic rule of a CA that spreads its information (infection) across local and distant neighbors by representing spread throughout the CA lattice.

■ 2.3 Nonuniform Temporally Stochastic Cellular Automata

To incorporate the given features, we ease some of the limitations of traditional CAs. As a result, we propose a new variant of CA, NTSCA, as a natural choice to develop the epidemic spread model.

To represent the association of the entire world, the model is designed over a two-dimensional grid with n cells having periodic boundary conditions. That is, the lattice can be considered as $\mathcal{L} = \mathbb{Z} / n\mathbb{Z}$, $n \in \mathbb{N}$. In this paper, we consider two-state CAs, so $S = \{0, 1\}$. Each cell represents a block of a specific geographic region, assuming state 1 in the event of infection and state 0 in the absence of infection. The CA follows the Moore neighborhood condition; that is, a cell takes the nearest cells—top-left $\nearrow$, top $\uparrow$, top-right $\nearrow$, left $\leftarrow$, right $\rightarrow$, down-left $\swarrow$, down $\downarrow$ and down-right $\searrow$ cells—and itself $\diamond$ as its neighbors. Additionally, some cells may have a small number of distant neighbors, which can be any other cell in the lattice, indicating the nonlocal spread of the viral infection caused by the long-distance journey of some infected individuals. To illustrate this circumstance, we consider the possibility of each cell having a distant neighbor $\star$ with a probability of p (Figure 1). We consider the symmetric relationship: if a cell i has a distant neighbor at cell x , then for

cell x also, cell i is its distant neighbor. The neighborhood vector

$$\mathcal{M} = (\vec{v}_\star, \vec{v}_\nwarrow, \vec{v}_\downarrow, \vec{v}_\swarrow, \vec{v}_\rightarrow, \vec{v}_\circ, \vec{v}_\leftarrow, \vec{v}_\nearrow, \vec{v}_\uparrow, \vec{v}_\searrow),$$

where $\vec{v}_\star$ can be any cell of the lattice. A configuration is the collection of states of all the cells at a specific moment. If x is a configuration, then $x = (x_{i,j})$ $i, j \in \mathcal{L}$, where $x_{i,j}$ is the state of cell $(i, j) \in \mathcal{L}$. The set of all configurations is denoted by $\mathcal{S}^{\mathcal{L}}$.

To update its current state, a cell considers the states of itself, its eight nearest neighbors and a distant neighbor. A 10-tuple is provided as input to the rule $\mathcal{R}$:

$$\langle s_\star, s_\nwarrow, s_\downarrow, s_\swarrow, s_\rightarrow, s_\circ, s_\leftarrow, s_\nearrow, s_\uparrow, s_\searrow \rangle.$$

The elements of the tuple represent the 10 neighbors from $\mathcal{M}$ of a cell. This tuple can be considered as the rule min term (RMT). It is obvious that each of the states

$$s_\star, s_\nwarrow, s_\downarrow, s_\swarrow, s_\rightarrow, s_\circ, s_\leftarrow, s_\nearrow, s_\uparrow, s_\searrow \in \{0, 1\}.$$

Each RMT can be represented using its equivalent decimal number

$$r = 2^9 \cdot s_\star + 2^8 \cdot s_\nwarrow + 2^7 \cdot s_\downarrow + 2^6 \cdot s_\swarrow + 2^5 \cdot s_\rightarrow + 2^4 \cdot s_\circ + 2^3 \cdot s_\leftarrow + 2^2 \cdot s_\nearrow + 2 \cdot s_\uparrow + s_\searrow.$$

Hence, the possible number of RMTs is $2^{10} = 1024$, and $\mathcal{R}[r]$ denotes the next state for RMT r under rule $\mathcal{R}$, that is,

$$\mathcal{R}(s_\star, s_\nwarrow, s_\downarrow, s_\swarrow, s_\rightarrow, s_\circ, s_\leftarrow, s_\nearrow, s_\uparrow, s_\searrow).$$

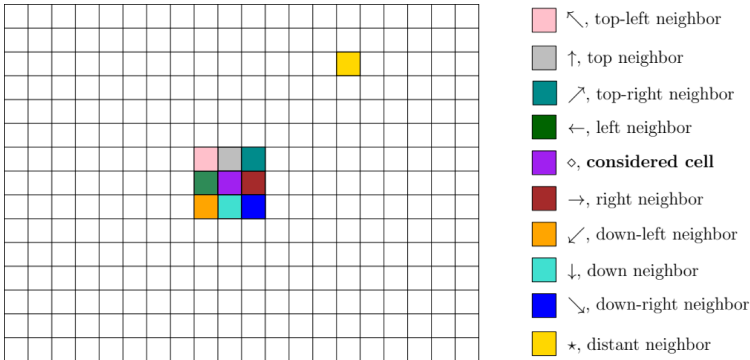


Figure 1. A cell and its neighborhood: neighborhood structure.

The transition rule for an NTSCA changes with time. However, at a specific time stamp, all the cells in the grid use the same transition function. A TSCA can be defined as follows.

Definition 1. [11] Assume a CA uses two local rules f and g , where f is the default rule, which is applied uniformly over the grid, whereas g is treated as a noise rule and applied with a noise rate $\tau \in [0, 1]$. Hence, rule f is applied at probability $1 - \tau$. Such CAs are TSCAs. We identify the TSCA with the notation $(f, g)[\tau]$.

In this model, we consider the applied TSCA changes over time. Assume that for the time frame between time steps T_0 and T' , the TSCAs are applied according to the sequence:

$$\langle ((f_{t_i}, g_{t_i})[\tau_{t_i}])|_{t_i}, \dots, ((f_{t_{i+k}}, g_{t_{i+k}})[\tau_{t_{i+k}}])|_{t_{i+k}}, \dots, ((f_{t_j}, g_{t_j})[\tau_{t_j}])|_{t_j} \rangle,$$

where $((f_{t_i}, g_{t_i})[\tau_{t_i}])$ is applied for the t_i period of time starting from T_0 and continuing subsequently. To make changes synchronously at each time step, one of the rules f_i or g_i is used:

$$s_{i,j}^{t+1} = \begin{cases} f_i(s_{p,q}^t, s_{i-1,j-1}^t, s_{i,j-1}^t, s_{i+1,j-1}^t, s_{i-1,j}^t, & \text{with probability } 1 - \tau_i \\ s_{i,j}^t, s_{i+1,j}^t, s_{i-1,j+1}^t, s_{i,j+1}^t, s_{i+1,j+1}^t) \\ g_i(s_{p,q}^t, s_{i-1,j-1}^t, s_{i,j-1}^t, s_{i+1,j-1}^t, s_{i-1,j}^t, & \text{with probability } \tau_i. \\ s_{i,j}^t, s_{i+1,j}^t, s_{i-1,j+1}^t, s_{i,j+1}^t, s_{i+1,j+1}^t) \end{cases}$$

Here the cells' locations are denoted using i, j, p and q ; $s_{i,j}^t$ indicates the present state (at time t) of the cell (i, j) , whereas $s_{i,j}^{t+1}$ denotes the state of the cell (i, j) for the next time step $t + 1$. To indicate the distant neighbor, we use the symbol $\star$. The location of $\star$ is defined as (p, q) . Cell $(i - 1, j - 1)$ is indicated as $\swarrow$, $(i, j - 1)$ is indicated as $\downarrow$, and so on.

3. Design of Rules

Any highly contagious virus has the significant behavior that its propagation is very much unpredictable; one evidence of this unpredictability is the growth in the number of *asymptomatic* patients. We have already mentioned that the local rules need to be chaotic in nature to mimic this behavior. Two rules, f and g , can be used in our model to update a cell at a given time step. Here, f represents the *virus propagation*, whereas g represents the *immunity* and is applied with probability τ . Hence, τ can be considered as the rate of immunity of the model. The sequence of TSCAs is:

$$\langle ((f_{t_i}, g_{t_i})[\tau_{t_i}])|_{t_i}, \dots, ((f_{t_{i+k}}, g_{t_{i+k}})[\tau_{t_{i+k}}])|_{t_{i+k}}, \dots, ((f_{t_j}, g_{t_j})[\tau_{t_j}])|_{t_j} \rangle,$$

and the model is named as NTSCA.

There is an enormous number, $2^{2^{10}}$, of conceivable two-dimensional two-state 10-neighborhood CAs. Therefore, it is very difficult to conduct a thorough search of the rule space to identify the best

possible sequence of chaotic rules that can accurately represent every behavior of a highly contagious disease. In order to construct an NTSCA that meets the criteria listed in Section 2, we first identify potential candidate rules that may be employed in TSCAs. As mentioned, these CA rules need to be chaotic when defined over an infinite lattice.

The update of each cell depends on its neighbors. Hence, if some neighbors change their states, the changes may or may not affect the cell under consideration. That implies the cell is sensitive or insensitive to its neighbors. Therefore, the unpredictability of the next configuration depends on the fact that the cells are sensitive to their neighbors and the initial configuration. In fact, a CA can be unpredictable if during evolution the CA gradually destroys information about the initial configuration [13]. Such CAs can be considered as chaotic. We now discuss the *information flow* of a CA, which acts as a measure of unpredictability in CAs. Information flow consists of two parameters: *information propagation* and *information cooking*.

3.1 Information Propagation

Since a virus can start spreading from a single geographic place, it is important to guarantee that in our model, even the smallest change in information from the first configuration spreads across the CA and impacts all subsequent configurations. A cell's state in a CA, however, may vary based on the states of its neighbors. It is therefore preferable that the cells in the rule be sensitive to their neighbors, so that changes in their state cause an update of the state in the cell being considered. As a result, throughout the course of evolution, a cell's infection (information) spreads to its neighbors and affects a significant number of cells. We now consider the rule min terms (RMTs) to comprehend and quantify this information propagation.

Definition 2. A set of RMTs is called k -equivalent or $\mathcal{E}^k$ if the values of all their neighbors are invariant except the k^{th} neighbor, where $k \in \mathcal{M}$. Mathematically,

$$\mathcal{E}_i^k = \{a_1 a_2 \cdots a_{k-1} x a_{k+1} \cdots a_m \in \mathcal{S}^m \mid x \in \mathcal{S}\}.$$

Here, i is the decimal equivalent of

$$a_1 a_2 \cdots a_{k-1} a_{k+1} \cdots a_m$$

and $|\mathcal{M}| = m$.

So $\mathcal{E}_i^k = \{j, 2^k + j\}$ for the proposed CAs where

$$j = 2^{k+1} \cdot \frac{i}{2^k} + (i \bmod 2^k).$$

Ten sets of equivalent RMTs exist, with each having $2^9 = 512$ members:

1. $\mathcal{E}_i^* = \{xa_{\searrow}a_{\downarrow}a_{\swarrow}a_{\rightarrow}a_{\circ}a_{\leftarrow}a_{\nearrow}a_{\nwarrow}\} \in \{0, 1\}^{10} \mid x \in \{0, 1\}, 0 \leq i \leq 511$
2. $\mathcal{E}_i^{\searrow} = \{a_{\star}xa_{\downarrow}a_{\swarrow}a_{\rightarrow}a_{\circ}a_{\leftarrow}a_{\nearrow}a_{\nwarrow}\} \in \{0, 1\}^{10} \mid x \in \{0, 1\}, 0 \leq i \leq 511$
3. $\mathcal{E}_i^{\downarrow} = \{a_{\star}a_{\searrow}xa_{\swarrow}a_{\rightarrow}a_{\circ}a_{\leftarrow}a_{\nearrow}a_{\nwarrow}\} \in \{0, 1\}^{10} \mid x \in \{0, 1\}, 0 \leq i \leq 511$
4. $\mathcal{E}_i^{\swarrow} = \{a_{\star}a_{\searrow}a_{\downarrow}xa_{\rightarrow}a_{\circ}a_{\leftarrow}a_{\nearrow}a_{\nwarrow}\} \in \{0, 1\}^{10} \mid x \in \{0, 1\}, 0 \leq i \leq 511$
5. $\mathcal{E}_i^{\rightarrow} = \{a_{\star}a_{\searrow}a_{\downarrow}a_{\swarrow}xa_{\rightarrow}a_{\circ}a_{\leftarrow}a_{\nearrow}a_{\nwarrow}\} \in \{0, 1\}^{10} \mid x \in \{0, 1\}, 0 \leq i \leq 511$
6. $\mathcal{E}_i^{\circ} = \{a_{\star}a_{\searrow}a_{\downarrow}a_{\swarrow}a_{\rightarrow}xa_{\leftarrow}a_{\nearrow}a_{\nwarrow}\} \in \{0, 1\}^{10} \mid x \in \{0, 1\}, 0 \leq i \leq 511$
7. $\mathcal{E}_i^{\leftarrow} = \{a_{\star}a_{\searrow}a_{\downarrow}a_{\swarrow}a_{\rightarrow}a_{\circ}xa_{\leftarrow}a_{\nearrow}a_{\nwarrow}\} \in \{0, 1\}^{10} \mid x \in \{0, 1\}, 0 \leq i \leq 511$
8. $\mathcal{E}_i^{\nearrow} = \{a_{\star}a_{\searrow}a_{\downarrow}a_{\swarrow}a_{\rightarrow}a_{\circ}a_{\leftarrow}xa_{\nearrow}a_{\nwarrow}\} \in \{0, 1\}^{10} \mid x \in \{0, 1\}, 0 \leq i \leq 511$
9. $\mathcal{E}_i^{\nwarrow} = \{a_{\star}a_{\searrow}a_{\downarrow}a_{\swarrow}a_{\rightarrow}a_{\circ}a_{\leftarrow}a_{\nearrow}xa_{\nwarrow}\} \in \{0, 1\}^{10} \mid x \in \{0, 1\}, 0 \leq i \leq 511$
10. $\mathcal{E}_i^{\nwarrow} = \{a_{\star}a_{\searrow}a_{\downarrow}a_{\swarrow}a_{\rightarrow}a_{\circ}a_{\leftarrow}a_{\nearrow}a_{\nwarrow}x\} \in \{0, 1\}^{10} \mid x \in \{0, 1\}, 0 \leq i \leq 511$

Now assume that the state of the cell, denoted by the symbol $\diamond$, has changed from $s_{\diamond}$ to $s'_{\diamond}$. We have to evaluate its impact on the remaining nine adjacent cells. When a cell's neighbors update their states, we consider that the information propagates in those directions. Consider that the RMTs $\mathcal{E}_i^{\downarrow}$ possess various next state values, where $0 \leq i \leq 511$; that is,

$$\mathcal{R}[a_{\star}a_{\searrow}a_{\downarrow}a_{\swarrow}a_{\rightarrow}a_{\circ}a_{\leftarrow}a_{\nearrow}a_{\nwarrow}0] \neq \mathcal{R}[a_{\star}a_{\searrow}a_{\downarrow}a_{\swarrow}a_{\rightarrow}a_{\circ}a_{\leftarrow}a_{\nearrow}a_{\nwarrow}1],$$

where

$$a_{\star}a_{\searrow}a_{\downarrow}a_{\swarrow}a_{\rightarrow}a_{\circ}a_{\leftarrow}a_{\nearrow}a_{\nwarrow} \in \{0, 1\}^9.$$

This indicates that any new information added to a cell always spreads to its upper-left neighbor. However, if this is the case for some i , then the information propagating in the upper-left ($\nwarrow$) direction will not move at a constant rate of 100%. Instead, information propagation will depend on how often the RMTs for all i have different next state values. To obtain the information propagation of any neighbor k , $k \in \mathcal{M}$, we define a binary function

$$\delta_i^k : \mathcal{E}_i^k \times \mathcal{E}_i^k \longrightarrow \{0, 1\}$$

where

$$\delta_i^k(r, s) = \begin{cases} 1 & \text{if } \mathcal{R}[r] \neq \mathcal{R}[s] \\ 0 & \text{otherwise.} \end{cases}$$

Here, $r, s \in \mathcal{E}_i^k$, $r \neq s$ and $0 \leq i \leq d^{m-1} - 1$ and $|\mathcal{S}| = d = 2$. In this paper, we consider the number of elements in the set $\mathcal{E}_i^k = 2$, so the

effective number of times $\mathcal{E}_i^k$ gets updated is half of $\delta_i^k(r, s)$. That is,

$$\lambda_i^k = \frac{1}{d(d-1)} \sum_{r,s \in \mathcal{E}_i^k, r \neq s} \delta_i^k(r, s).$$

Consequently, due to a change in the current cell, the overall possibility of information propagation to the k^{th} neighboring cell is

$$\Lambda_p^k = \frac{1}{d^{m-1}} \sum_{i=0}^{d^{m-1}-1} \lambda_i^k,$$

where $m = |\mathcal{M}| = 10$. The information propagation to the k^{th} neighbor is measured by this value. If RMTs of $\mathcal{E}_i^k$ have different next state values for all i , then Λ_p^k approaches the maximum value 1.

We only use CAs with information propagating in all directions in order to represent the high infection rate of highly contagious viruses. Additionally, to ensure that the system is impartial, we assume that the CAs do not prefer any specific state. Such CA rules are called *balanced* because the total number of RMTs with each subsequent state value is equal.

3.2 Information Cooking

In the previous section, we discussed how updating a cell $\diamond$ affects the other neighboring cells. We now look at how the update impacts the cell itself. It goes without saying that when a cell's neighbors change in response to a cell's update, the cell $\diamond$ will update again in the next time step in response to those updates. For a CA to be unpredictable, there must be this type of dependency on the neighbors. Information cooking from the k^{th} neighbor denotes the possibility of updating cell $\diamond$ in the subsequent time step in response to the update of s_o that has affected the k^{th} neighbor. To calculate this, we need to define another equivalent set, named the K -set.

Definition 3. A set of RMTs corresponding to an RMT r is called the K -set(r) or $\Omega^k(r)$ if the values of only the k^{th} neighbor are invariant, where $k \in \mathcal{M}$. Mathematically,

$$\Omega^k(r) = \{a'_1 a'_2 \cdots a'_{k-1} a_k a'_{k+1} \cdots a'_m \in \mathcal{S}^m \mid r\}$$

is the decimal equivalent of

$$a_1 a_2 \cdots a_{k-1} a_k a_{k+1} \cdots a_m, a'_i \neq a_i,$$

where $i \neq k$, $a'_i \in \mathcal{S}$ and $|\mathcal{M}| = m$.

Let RMT r be the decimal equivalent of

$$a_{\star} a_{\searrow} a_{\downarrow} a_{\swarrow} a_{\rightarrow} a_{\circ} a_{\leftarrow} a_{\nearrow} a_{\uparrow} a_{\nwarrow}.$$

Because 10 neighbors exist, there are 10 such K -sets:

1. $\Omega^{\star}(r) = \{a_{\star}a'_{\searrow}a'_{\swarrow}a'_{\rightarrow}a'_{\circ}a'_{\nearrow}a'_{\uparrow}a'_{\nwarrow}\} \in \{0, 1\}^{10} \mid 0 \leq r \leq 511$
2. $\Omega^{\searrow}(r) = \{a'_{\star}a_{\searrow}a'_{\swarrow}a'_{\rightarrow}a'_{\circ}a'_{\nearrow}a'_{\uparrow}a'_{\nwarrow}\} \in \{0, 1\}^{10} \mid 0 \leq r \leq 511$
3. $\Omega^{\swarrow}(r) = \{a'_{\star}a_{\searrow}a'_{\swarrow}a'_{\rightarrow}a'_{\circ}a'_{\nearrow}a'_{\uparrow}a'_{\nwarrow}\} \in \{0, 1\}^{10} \mid 0 \leq r \leq 511$
4. $\Omega^{\rightarrow}(r) = \{a'_{\star}a'_{\searrow}a'_{\swarrow}a_{\rightarrow}a'_{\circ}a'_{\nearrow}a'_{\uparrow}a'_{\nwarrow}\} \in \{0, 1\}^{10} \mid 0 \leq r \leq 511$
5. $\Omega^{\circ}(r) = \{a'_{\star}a'_{\searrow}a'_{\swarrow}a'_{\rightarrow}a_{\circ}a'_{\nearrow}a'_{\uparrow}a'_{\nwarrow}\} \in \{0, 1\}^{10} \mid 0 \leq r \leq 511$
6. $\Omega^{\nearrow}(r) = \{a'_{\star}a'_{\searrow}a'_{\swarrow}a'_{\rightarrow}a'_{\circ}a_{\nearrow}a'_{\uparrow}a'_{\nwarrow}\} \in \{0, 1\}^{10} \mid 0 \leq r \leq 511$
7. $\Omega^{\uparrow}(r) = \{a'_{\star}a'_{\searrow}a'_{\swarrow}a'_{\rightarrow}a'_{\circ}a'_{\nearrow}a_{\uparrow}a'_{\nwarrow}\} \in \{0, 1\}^{10} \mid 0 \leq r \leq 511$
8. $\Omega^{\nwarrow}(r) = \{a'_{\star}a'_{\searrow}a'_{\swarrow}a'_{\rightarrow}a'_{\circ}a'_{\nearrow}a'_{\uparrow}a_{\nwarrow}\} \in \{0, 1\}^{10} \mid 0 \leq r \leq 511$
9. $\Omega^{\nearrow}(r) = \{a'_{\star}a'_{\searrow}a'_{\swarrow}a'_{\rightarrow}a'_{\circ}a'_{\nearrow}a'_{\uparrow}a'_{\nwarrow}\} \in \{0, 1\}^{10} \mid 0 \leq r \leq 511$
10. $\Omega^{\nwarrow}(r) = \{a'_{\star}a'_{\searrow}a'_{\swarrow}a'_{\rightarrow}a'_{\circ}a'_{\nearrow}a'_{\uparrow}a_{\nwarrow}\} \in \{0, 1\}^{10} \mid 0 \leq r \leq 511$

Suppose that the states $s_{\star}, s_{\searrow}, s_{\swarrow}, s_{\rightarrow}, s_{\circ}, s_{\nearrow}, s_{\uparrow}$ and $s_{\nwarrow}$ are updated to $s'_{\star}, s'_{\searrow}, s'_{\swarrow}, s'_{\rightarrow}, s'_{\circ}, s'_{\nearrow}, s'_{\uparrow}$ and $s'_{\nwarrow}$, respectively. If

$$\begin{aligned} \mathcal{R}(s_{\star}, s_{\searrow}, s_{\swarrow}, s_{\rightarrow}, s_{\circ}, s_{\nearrow}, s_{\uparrow}, s_{\nwarrow}) &\neq \\ \mathcal{R}(s'_{\star}, s'_{\searrow}, s'_{\swarrow}, s'_{\rightarrow}, s'_{\circ}, s'_{\nearrow}, s'_{\uparrow}, s'_{\nwarrow}), \end{aligned}$$

then it indicates that updates of the cell $\diamond$ itself and its other neighbors (except $s_{\nwarrow}$) have an impact on the considering cell. This scenario may be considered an information flow from other adjacent neighbors to the cell. We consider this as information cooking due to the neighbor $\nwarrow$. To measure the amount of impact due to the k^{th} neighbor, we define a function $\Delta_r^k: \Omega^k(r) \rightarrow \{0, 1\}$ by

$$\Delta_r^k(s) = \begin{cases} 1 & \text{if } \mathcal{R}(r) \neq \mathcal{R}(s), s \in \Omega^k(r) \\ 0 & \text{otherwise.} \end{cases}$$

The number of times a cell $\diamond$ changes in response to changes of its neighbors can now be determined by using the parameter η_r^k :

$$\eta_r^k = \frac{1}{(d-1)^{m-1}} \sum_{s \in \Omega^k(r)} \Delta_r^k(s).$$

Consequently, the overall possibility that the cell $\diamond$ will change when the neighboring cells change can be stated as

$$\Lambda_c^k = \frac{1}{d^m} \sum_{r=0}^{d^m-1} \eta_r^k.$$

Here, $m = |\mathcal{M}| = 10$ and $\Lambda_c^{\nwarrow}$ represents the amount of information cooking for the cell $\diamond$ due to neighbor $\nwarrow$ and so on. The overall information flow to the k^{th} neighbor, that is, the possibility of disturbing

the k^{th} neighbor due to a change in the current cell, is

$$\mathbb{K} = (\Lambda_p^k, \max(\forall_{j \neq k} \Lambda_p^j, \Lambda_c^k)).$$

Furthermore, the external factors that prevent the spreading of disease also need to be incorporated in the transition rules. We employ partial blocking of information flow in the transition rules to achieve this.

3.3 Partial Information Blocking

There is a probability that information from one side of the lattice does not reach the other when the rate of information propagation and information cooking in all directions is not 100%; that is, $\forall k, \Lambda_p^k \neq 1$ and $\forall k, \Lambda_c^k \neq 1$, where Λ_p^k and Λ_c^k are the information propagation and cooking in the k^{th} direction, respectively. That implies some areas of the lattice are (partially) blocking the transfer of information (infection). In other words, the infected cells cannot communicate with other cells. Therefore, the blocking opposes the spreading of infections from outside. Similarly, if a region is already infected, the virus may not spread to nearby areas.

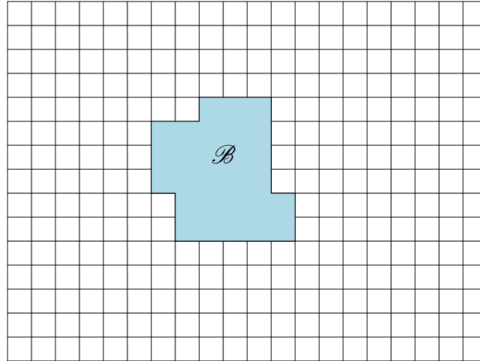


Figure 2. An existence block $\mathcal{B}$ containing a number of cells in the lattice $\mathcal{L}$.

Let us analyze Figure 2: any cell $x \in \mathcal{B}$ does not change its state by the changes of its neighbors. This type of sublattice $\mathcal{B} \subset \mathcal{L}$ is referred to as a *block*. In such cases, the rule keeps certain RMTs that depend only on itself and are independent of its neighbors. These RMTs are referred to as *self-replicating* RMTs. Let

$$\mathcal{R}(s_{\star}, s_{\searrow}, s_{\downarrow}, s_{\swarrow}, s_{\rightarrow}, s_{\circ}, s_{\leftarrow}, s_{\nearrow}, s_{\uparrow}, s_{\nwarrow}) = s_{\circ},$$

where $s_{\star}, s_{\searrow}, s_{\downarrow}, s_{\swarrow}, s_{\rightarrow}, s_{\circ}, s_{\leftarrow}, s_{\nearrow}, s_{\uparrow}, s_{\nwarrow}$ denote the neighboring cells' states of cell j . Then the RMT

$$\langle s_{\star}, s_{\searrow}, s_{\downarrow}, s_{\swarrow}, s_{\rightarrow}, s_{\circ}, s_{\leftarrow}, s_{\nearrow}, s_{\uparrow}, s_{\nwarrow} \rangle$$

is denoted as a *self-replicating* RMT.

Similar to this, if a cell has k ($k \in \mathcal{M}$) neighbors that are outside of the block $\mathcal{B}$, the rule has to be formed in such a manner that the cell is independent of each of these k neighbors for certain particular values of the RMTs. Let the blocking function be $b: \mathcal{B} \mapsto \{0, 1\}$. Then, for each cell $x \in \mathcal{B}$ with the current state s ,

$$b(x) = s \begin{cases} \text{if } \mathcal{R}(a_1 a_2 \cdots a_{k-1} s a_{k+1} \cdots a_{m-1}) = s, \\ \text{where } \tilde{a}_1, \dots, \tilde{a}_k \\ \text{if } \mathcal{R}(a_1 \cdots a_{l-1} 0 a_{l+1} \cdots a_{k-1} s a_{k+1} \cdots a_{m-1}) = \\ \quad \mathcal{R}(a_1 a \cdots a_{l-1} 1 a_{l+1} \cdots a_{k-1} s a_{k+1} \cdots a_{m-1}) = s, \\ \text{where } \tilde{a}_1, \dots, \tilde{a}_{l-1} \tilde{a}_{l+1} \cdots \tilde{a}_{k-1}, \tilde{a}_{k+1}, \tilde{a}_{m-1} \in \mathcal{B}, \text{ but } \tilde{a}_l \notin \mathcal{B}. \end{cases}$$

Here, $a_1, a_2, \dots, a_{k-1}, a_{k+1}, \dots, a_{m-1}$ are the current states of the $m-1$ neighbors, which are located at cells $\tilde{a}_1, \dots, \tilde{a}_{k-1}, \tilde{a}_{k+1}, \tilde{a}_{m-1}$, respectively.

Therefore, each cell's state remains unchanged when a group of cells called a block $\mathcal{B}$ is formed. If we enable certain cells inside $\mathcal{B}$ that have neighbors outside $\mathcal{B}$ to be dependent on that neighbor, this block may also be diminished. Assume that some cells in block $\mathcal{B}$ are dependent on their neighbors outside of $\mathcal{B}$; that is, a cell x , and some of its neighbors $\tilde{x}_\star, \tilde{x}_\searrow, \tilde{x}_\downarrow, \tilde{x}_\swarrow, \tilde{x}_\rightarrow, \tilde{x}_\leftarrow, \tilde{x}_\nearrow, \tilde{x}_\uparrow \in \mathcal{B}$, but $\tilde{x}_\nwarrow \notin \mathcal{B}$, then $\mathcal{B}$ may also be diminished. For example, the cell x changes its state depending on its top-left $\nwarrow$ neighbor if

$$\begin{aligned} \mathcal{R}(\tilde{x}_\star, \tilde{x}_\searrow, \tilde{x}_\downarrow, \tilde{x}_\swarrow, \tilde{x}_\rightarrow, \tilde{x}_\circ, \tilde{x}_\leftarrow, \tilde{x}_\nearrow, \tilde{x}_\uparrow, 0) &\neq \\ \mathcal{R}(\tilde{x}_\star, \tilde{x}_\searrow, \tilde{x}_\downarrow, \tilde{x}_\swarrow, \tilde{x}_\rightarrow, \tilde{x}_\circ, \tilde{x}_\leftarrow, \tilde{x}_\nearrow, \tilde{x}_\uparrow, 1). \end{aligned}$$

Assume

$$\mathcal{R}(\tilde{x}_\star, \tilde{x}_\searrow, \tilde{x}_\downarrow, \tilde{x}_\swarrow, \tilde{x}_\rightarrow, \tilde{x}_\circ, \tilde{x}_\leftarrow, \tilde{x}_\nearrow, \tilde{x}_\uparrow, 0) = x_\circ;$$

that implies the cell x updates its state, which in turn reduces the block when the state of neighboring cell $\tilde{x}_\nwarrow$ switches to 1. Therefore, our aim is to define a CA such that it can develop blocks that may also be diminished under certain conditions.

3.4 Margin of Error

The proposed NTSCA is a stochastic CA-based model that uses a sequence of TSCAs. Each TSCA consists of two rules, f and g . Rule f is treated as the default rule, and a noise rate g is applied with a probability τ . This shows that each cell is stochastically updated, and the result may vary for different runs. This variation of results in different runs is expressed as the *margin of error*, where the maximum expected difference between an estimated sample parameter and the true parameter is noted [14].

In our model, a cell follows a 10-neighborhood structure, which implies that the information flows in every direction. To obtain approximate results, the differences need to be recorded and a margin

of error needs to be calculated for each rule in every direction. For our model, consider the sample size $m = 2^9 = 512$ and estimate the margin of error as:

$$\text{margin of error} = Z_{n/2} \left(\frac{\sigma}{\sqrt{m}} \right),$$

where the variance σ is denoted as

$$\sigma = \sqrt{\frac{\sum (x_i - \bar{x})^2}{(m-1)}}.$$

Here, α_i α are denoted as the i^{th} sample efficiency and mean efficiency, respectively. We consider the confidence level of the experiments as 95% and $Z_{n/2} = 1.96$.

4. Case Study: Implementation of COVID-19 Simulation

Since COVID-19 started spreading from a small city in China (as per report), our proposed model considers that initially every cell is set to 0, except for a single one. Our main objective is to simulate the spread of COVID-19 in India and use real-time data to validate the simulation results. In order to mimic the spread of COVID-19, we employ a sequence of TSCA rules to develop an NTSCA. This is similar to the COVID-19 epidemic, in which many waves of infection and mutations of the coronavirus were noticed. For the duration of the COVID-19 pandemic, our proposed NTSCA follows these steps:

$$\langle (f_1, g_1)[0.0] \rangle_{30}, \langle (f_1, g_1)[0.3] \rangle_{50}, \\ \langle (f_2, g_2)[0.3] \rangle_{30}, \langle (f_3, g_3)[0.3] \rangle_{50} \rangle.$$

Initially, a TSCA $(f_1, g_1)[0]$ is applied for 30 consecutive steps, then $(f_1, g_1)[0.3]$ is applied for the following 50 steps, $(f_2, g_2)[0.3]$ is applied for another 30 consecutive steps and $(f_3, g_1)[0.3]$ is applied for 50 steps. After that, $((f_2, g_2)[0.3])$ is applied to stabilize the system for 100 consecutive steps (as default TSCA). The rules are defined in the following.

$$\text{Rule } f_1 = \begin{cases} 1 \text{ with probability } q_1 \times 0.05 & \text{where } q_1 = \sum_{\substack{i-1 \leq l \leq i+1, \\ j-1 \leq m \leq j+1}} C_{(l,m)}^1 \text{ if } s_{i,j}^t = 0 \\ s_{i,j}^t & \text{otherwise} \end{cases}$$

$$\text{Rule } f_2 = \begin{cases} 1 & \text{if } s_{i,j}^t = 0 \text{ and } \sum_{\substack{i-1 \leq l \leq i+1, \\ j-1 \leq m \leq j+1}} C_{(l,m)}^1 > 4 \\ s_{i,j}^t & \text{otherwise} \end{cases}$$

$$\begin{aligned}
\text{Rule } f_3 &= \begin{cases} 1 \text{ with probability } q_1 \times 0.1 & \text{where } q_1 = \sum_{\substack{i-1 \leq l \leq i+1, \\ j-1 \leq m \leq j+1}} C_{(l,m)}^1 \text{ if } s_{i,j}^t = 0 \\ s_{i,j}^t & \text{otherwise} \end{cases} \\
\text{Rule } g_1 &= \begin{cases} 0 \text{ with probability } q_0 \times 0.1 & \text{where } q_0 = \sum_{\substack{i-1 \leq l \leq i+1, \\ j-1 \leq m \leq j+1}} C_{(l,m)}^0 \text{ if } s_{i,j}^t = 1 \\ s_{i,j}^t & \text{otherwise} \end{cases} \\
\text{Rule } g_2 &= \begin{cases} 0 \text{ with probability } q_0 \times 0.01 & \text{where } q_0 = \sum_{\substack{i-1 \leq l \leq i+1, \\ j-1 \leq m \leq j+1}} C_{(l,m)}^0 \text{ if } s_{i,j}^t = 1 \\ s_{i,j}^t & \text{otherwise} \end{cases}
\end{aligned}$$

As mentioned, the TSCAs are utilized based on the spreading rate and recovery rate of the virus. Here, rule f indicates the infection rate, and rule g denotes the recovery rate. If recovery is viewed as an obstacle to the virus, then in the early stage, there is no impact of g (no recovery), and as a result, there is unrestrained spread of the disease. We use TSCA $(f_1, g_1)[0]$ for the first 30 steps of our simulation, where f_1 is applied with probability 1. During this period, the TSCA shows how the infection spreads without any constraints.

For the next consecutive 50 steps, $(f_1, g_1)[0.3]$ is applied, which indicates the application of recovery function g with a minimum rate 0.3. This is how we represent the first wave. Then, TSCA $(f_2, g_2)[0.3]$ is applied for the next consecutive 30 steps. This is thought to be the system's steady mode because both the rules f and g indicate a minimum rate of the viral spread and recovery, respectively. In the next 50 consecutive steps, $(f_3, g_3)[0.3]$ is applied, where the spreading rule f_3 has a higher spreading rate. This time frame is used to produce the second wave. To stabilize the simulation, $(f_2, g_2)[0.3]$ is applied for the duration. In this paper, we have considered the model to simulate up to the second wave of COVID-19 spread.

To verify our synthesized rules, we first calculate the information propagation in each direction of all rules. In Table 1, information propagation of each rule is shown along with Λ_p^k values, where $\forall k, k \in \mathcal{M}$. Each rule contains partial blocking and does not have maximum information flow in all directions.

As the rules are stochastically applied, this information propagation rate is not fixed, and a margin of error needs to be considered. Mean information propagation in every direction is reported in Table 1. Table 2 displays the margin of error of different rules in the model. It is found that the margin of error is very small for all the rules. Hence, during different runs, although the suggested model's

information propagation rate somewhat fluctuates, in every case a nonmaximal flow of information is maintained in every possible direction, and accordingly, a partial blocking of information happens.

Rule Name	$\Lambda_p^\star$	Λ_p°	$\Lambda_p^\downarrow$	$\Lambda_p^\leftarrow$	$\Lambda_p^\rightarrow$
f_1	0.21875	0.21875	0.2265625	0.234375	0.234375
f_2	0.0	0.13671875	0.13671875	0.13671875	0.13671875
f_3	0.228515625	0.240234375	0.208984375	0.224609375	0.224609375
g_1	0.244140625	0.216796875	0.228515625	0.228515625	0.216796875
g_2	0.01953125	0.01953125	0.01953125	0.01953125	0.01953125
g_3	0.138671875	0.107421875	0.130859375	0.126953125	0.111328125

Rule Name	Λ_p°	Λ_p°	Λ_p°	$\Lambda_p^\uparrow$	Λ_p°
f_1	0.60546875	0.1953125	0.22265625	0.21875	0.2109375
f_2	0.63671875	0.13671875	0.13671875	0.13671875	0.13671875
f_3	0.611328125	0.228515625	0.228515625	0.236328125	0.205078125
g_1	0.599609375	0.236328125	0.205078125	0.220703125	0.244140625
g_2	0.98046875	0.01953125	0.01953125	0.01953125	0.01953125
g_3	0.212890625	0.130859375	0.138671875	0.126953125	0.134765625

Table 1. Analysis of information propagation.

Rule Name	$\Lambda_p^\star$	Λ_p°	$\Lambda_p^\downarrow$	$\Lambda_p^\leftarrow$	$\Lambda_p^\rightarrow$
f_1	0.00129	0.00127	0.00134	0.00132	0.00132
f_2	0	0	0	0	0
f_3	0.00132	0.00134	0.00137	0.00127	0.00131
g_1	0.00134	0.00131	0.00134	0.00127	0.00132
g_2	0.00073	0.00073	0.00075	0.00075	0.000752
g_3	0.00095	0.00096	0.00103	0.00098	0.00097

Rule Name	Λ_p°	Λ_p°	Λ_p°	$\Lambda_p^\uparrow$	Λ_p°
f_1	0.00185	0.00142	0.00128	0.00132	0.00136
f_2	0	0	0	0	0
f_3	0.00185	0.00136	0.00130	0.00133	0.00133
g_1	0.00184	0.00136	0.00134	0.00137	0.00138
g_2	0.00079	0.00075	0.00076	0.00074	0.00076
g_3	0.00123	0.00101	0.00097	0.00105	0.00096

Table 2. Margin of error for calculating information propagation.

We calculate information cooking for each rule in every direction. Table 3 describes the information cooking rate of each rule with Λ_c^k values, where $\forall k \in \mathcal{M}$. Here, we observe that the information cooking rate for some rules (e.g., rules f_1 and g_2) is very high. It

indicates that the behavior of the model becomes unpredictable during evolution.

Rule Name	$\Lambda_c^\star$	$\Lambda_c^\succ$	$\Lambda_c^\downarrow$	$\Lambda_c^\swarrow$	$\Lambda_c^\rightarrow$
f_1	0.77476	0.77476	0.77476	0.77476	0.77476
f_2	0.39453	0.39453	0.40234	0.41016	0.44922
f_3	0.55062	0.55062	0.55062	0.55062	0.55062
g_1	0.54992	0.54992	0.54992	0.54992	0.54992
g_2	0.95573	0.95573	0.95573	0.95573	0.95573

Rule Name	Λ_c°	$\Lambda_c^\leftarrow$	$\Lambda_c^\nearrow$	$\Lambda_c^\uparrow$	$\Lambda_c^\nwarrow$
f_1	0.18016	0.77476	0.77476	0.77476	0.77476
f_2	0.53516	0.60547	0.52734	0.58203	0.56641
f_3	0.26938	0.55062	0.55062	0.55062	0.55062
g_1	0.27095	0.54992	0.54992	0.54992	0.54992
g_2	0.04238	0.95573	0.95573	0.95573	0.95573

Table 3. Information cooking of the rules.

Here also, the rules are stochastically applied, so this information cooking rate is also not fixed, and a margin of error needs to be considered. Table 4 shows the margin of error of different rules in the model. It is found that the margin of error is very small for all the rules. Hence, during different runs, although the suggested model’s information cooking rate somewhat fluctuates, in every case a nonmaximal cooking of information is maintained in every possible direction.

Rule Name	$\Lambda_c^\star$	$\Lambda_c^\succ$	$\Lambda_c^\downarrow$	$\Lambda_c^\swarrow$	$\Lambda_c^\rightarrow$
f_1	0.001592	0.001592	0.001592	0.001592	0.001592
f_2	0.00	0.00	0.00	0.00	0.00
f_3	0.001851	0.001851	0.001851	0.001851	0.001851
g_1	0.001832	0.001832	0.001832	0.001832	0.001832
g_2	0.000743	0.000743	0.000743	0.000743	0.000743

Rule Name	Λ_c°	$\Lambda_c^\leftarrow$	$\Lambda_c^\nearrow$	$\Lambda_c^\uparrow$	$\Lambda_c^\nwarrow$
f_1	0.18016	0.001283	0.001592	0.001592	0.001592
f_2	0.00	0.00	0.00	0.00	0.00
f_3	0.001313	0.001851	0.001851	0.001851	0.001851
g_1	0.001365	0.001832	0.001832	0.001832	0.001832
g_2	0.000721	0.000743	0.000743	0.000743	0.000743

Table 4. Margin of error for calculating information cooking of the rules.

4.1 Experimental Results of COVID-19 Simulation

Figure 3 displays the COVID-19 simulation results. Initially, all the cells' states are set to 0 and only one cell is set to 1. Here, we consider the color of the cell with state 0 as white and that of state 1 as black. A black cell is considered an infected cell, whereas the white cell represents an uninfected cell. Figure 3(a) shows the initial configuration of the simulation. The other figures (Figures 3(b) to 3(d)) show the intermediate configurations, that is, the infected and uninfected populations with regard to time (considering COVID-19 pandemic).

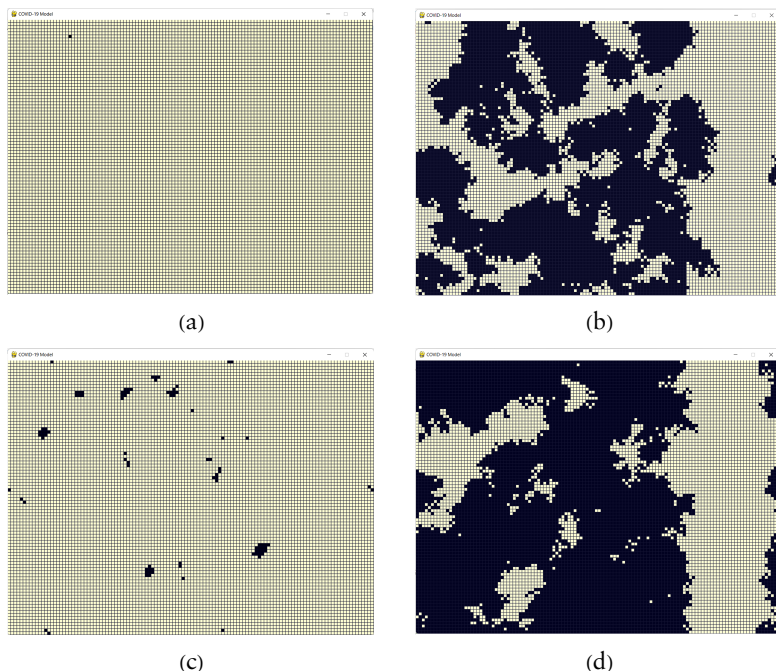


Figure 3. Simulation of COVID-19. (a) Initial configuration. (b) The first wave. (c) Low viral dissemination rate. (d) The second wave rising.

Figure 4 shows the graphical representation of the simulation, where Figures 4(a) and 4(b) depict the growth of infected cells (state: $0 \rightarrow 1$) and recovered cells (state: $1 \rightarrow 0$), respectively. Figure 4(d) shows the wave of the epidemic, that is, the number of 1s (infected cells) for different time periods. The simulation code is given in [15].

The real-time data of the COVID-19 epidemic spread in India is shown in Figure 5. We study the spreading dataset of COVID-19 of India (the dataset is from www.kaggle.com/datasets/imdevskp/coronavirus-report) up to the second wave. We consider three cases here: confirmed, recovered and the waves (first wave and second wave). We

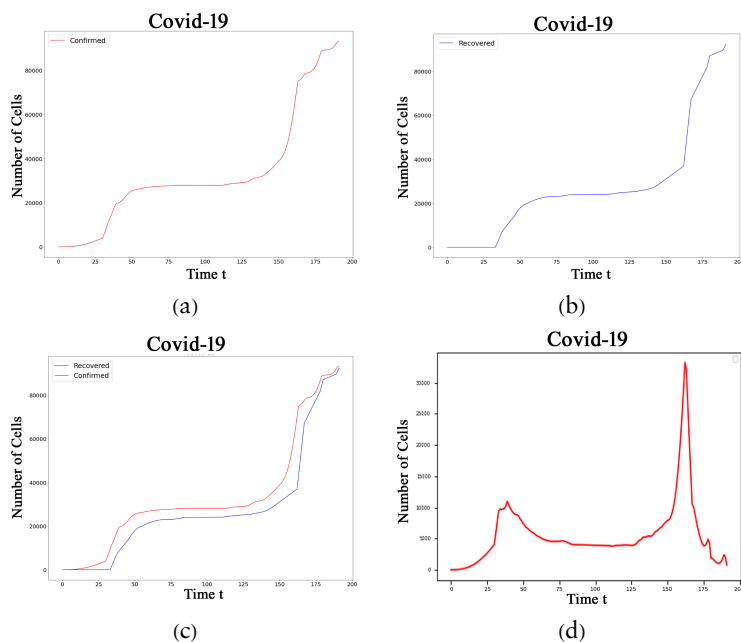


Figure 4. Simulation graphs of COVID-19 pandemic. (a) Confirmed cases over time. (b) Recovered cells over time. (c) Combination of (a) and (b). (d) The wave trajectory.

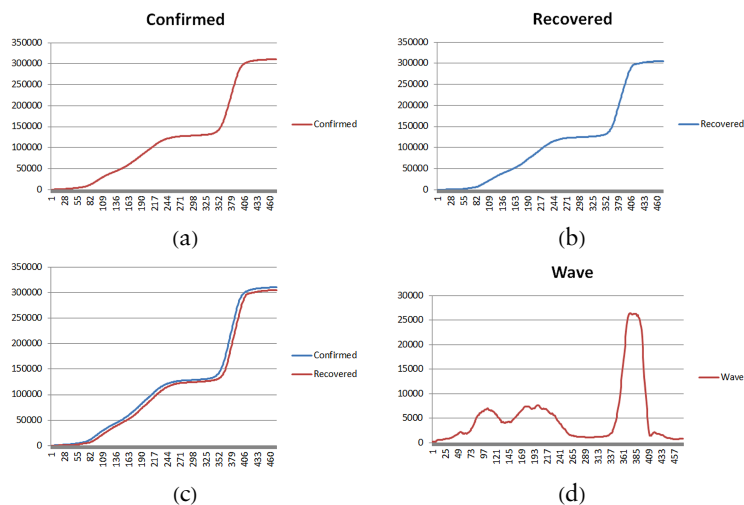


Figure 5. The graphs on real-time data of India. (a) Confirmed cases over time. (b) Recovered cells over time. (c) Combination of (a) and (b). (d) The wave trajectory.

preprocess the data approximately to fit the graphs. Then the simulation results are compared with the real-time data. By comparing Figures 4 and 5 with real-time datasets, we can see the accuracy and efficiency of the model.

Note that all the rules in the COVID-19 simulation are based on the spread of COVID in India up to the second wave. However, by changing the parameters, like probability of spreading, application time of a rule, the value of τ and the applied rules, for example, f_1, f_3, g_1, g_2 , and so on, we can model the spread of COVID-19 in any other geographical area. This implies that the proposed model is generic, whereas COVID-19 spreading in India is used as a case study to show the applicability of our model. In the next section, we consider another deadly virus to validate the accuracy of the proposed model.

5. Case Study: Implementation of SARS-2003 Simulation

SARS also started spreading from a small city in China, hence the initial configuration is considered the same as the previous case. Moreover, in a SARS epidemic the infection rate changes over time, along with mutations of the SARS virus. To imitate this, a sequence of TSCAs is considered to develop an NTSCA. The NTSCA considers the following sequence of TSCAs:

$$\begin{aligned} &\langle ((f_3, g_1)[0.0]) \mid_8, ((f_2, g_1)[0.0]) \mid_6, \\ &((f_3, g_1)[0.25]) \mid_7, ((f_2, g_3)[0.2]) \mid_{10}, \\ &((f_4, g_4)[0.35]) \mid_{20}, ((f_4, g_4)[0.6]) \mid_{60} \rangle. \end{aligned}$$

We use the rules f_1, f_2, f_3, g_1, g_2 from Section 4 and define the following new rules.

$$\begin{aligned} \text{Rule } f_4 &= \begin{cases} 1 \text{ with probability } e^{(q_1/12)-1} & \text{where } q_1 = \sum_{\substack{i-1 \leq l \leq i+1, \\ j-1 \leq m \leq j+1}} C_{(l,m)}^1 \text{ if } s_{i,j}^t = 0 \\ s_{i,j}^t & \text{otherwise} \end{cases} \\ \text{Rule } g_3 &= \begin{cases} 0 \text{ with probability } q_0 \times 0.15 & \text{where } q_0 = \sum_{\substack{i-1 \leq l \leq i+1, \\ j-1 \leq m \leq j+1}} C_{(l,m)}^0 \text{ if } s_{i,j}^t = 1 \\ s_{i,j}^t & \text{otherwise} \end{cases} \\ \text{Rule } g_4 &= \begin{cases} 0 \text{ with probability } \log_{10}(\sharp_0) / 3 & \text{where } \sharp_0 = \sum_{\substack{i-1 \leq l \leq i+1, \\ j-1 \leq m \leq j+1}} C_{(l,m)}^0 \text{ if } s_{i,j}^t = 1 \\ s_{i,j}^t & \text{otherwise} \end{cases} \end{aligned}$$

The nature of the two rules f and g is the same as previously assumed. At first $(f_3, g_1)[0.0]$ is applied for eight consecutive steps, where f_3 is applied with $\tau = 0$. Hence, at the initial stage, there is no recovery function. Thus the virus spreads unrestrained.

After 14 steps, another TSCA $(f_3, g_1)[0.25]$ is applied for the next seven steps. Then, $(f_2, g_3)[0.3]$ is applied for the following 10 steps to stabilize the spread of the virus because f_2 shows a minimal spread of virus. After that, TSCA $(f_4, g_4)[0.35]$ is applied for the next consecutive 20 steps. Here, rule f_4 represents that the growth of spreading is exponential with respect to the number of infected neighbors, whereas g_4 depicts the logarithmic growth of recovery rate. Here, rule g_4 applied with a low rate of $\tau = 0.35$ allows the required number of infected cells to be developed. Finally, to stabilize the simulation, rule g_4 is applied with a high probability of 0.6; that is, $(f_4, g_4)[0.6]$ is used for the remaining steps.

We synthesize the NTSCA rules by calculating the overall information flow, that is, information propagation, information cooking and margin of error of the defined rules. The mean value of information propagation in every direction of the rules is reported in Table 5 (and Table 1).

Rule Name	$\Lambda_p^\star$	$\Lambda_p^\rightarrow$	$\Lambda_p^\downarrow$	$\Lambda_p^\leftarrow$	$\Lambda_p^\rightarrow$
f_4	0.24632	0.24573	0.24539	0.24572	0.24464
g_3	0.18858	0.18852	0.18857	0.18862	0.18826
g_4	0.16301	0.16187	0.16113	0.16231	0.16188

Rule Name	Λ_p°	$\Lambda_p^\leftarrow$	$\Lambda_p^\rightarrow$	$\Lambda_p^\uparrow$	$\Lambda_p^\rightarrow$
f_4	0.46188	0.24436	0.24557	0.24567	0.24409
g_3	0.33266	0.18846	0.18727	0.18886	0.187996
g_4	0.79268	0.16188	0.16180	0.16173	0.16194

Table 5. Analysis of information propagation for the defined rules.

Here also, the information propagation rate is not fixed, and a margin of error needs to be considered. Tables 2 and 6 show the margin of error of different rules in every direction. It is found that the margin of error is very small for all the rules. Hence, during different runs, although the suggested model's information propagation rate somewhat fluctuates, in every case a nonmaximal propagation of information is maintained in every possible direction.

We calculate information cooking for every direction of each of these newly defined rules. Table 7 shows the information cooking rate of these rules with Λ_c^k values for each $k \in \mathcal{M}$ (see Table 3 for the other rules). Here, we can observe that the information cooking rate for some rules (e.g., rules f_1 and g_2) is very high. It indicates that the model shows chaotic behavior during evolution.

Rule Name	$\Lambda_p^\star$	$\Lambda_p^\succ$	$\Lambda_p^\downarrow$	$\Lambda_p^\swarrow$	$\Lambda_p^\rightarrow$
f_4	0.001417	0.001381	0.001365	0.001387	0.001365
g_3	0.001232	0.001207	0.001232	0.001238	0.001264
g_4	0.001232	0.001245	0.001142	0.001223	0.001151

Rule Name	Λ_p°	$\Lambda_p^\leftarrow$	$\Lambda_p^\nearrow$	$\Lambda_p^\uparrow$	$\Lambda_p^\nwarrow$
f_4	0.001846	0.001267	0.001321	0.001345	0.001360
g_3	0.001539	0.001322	0.001261	0.001263	0.001257
g_4	0.001439	0.001158	0.001200	0.001180	0.001200

Table 6. Margin of error for calculating information propagation of all the rules.

Rule Name	$\Lambda_c^\star$	$\Lambda_c^\succ$	$\Lambda_c^\downarrow$	$\Lambda_c^\swarrow$	$\Lambda_c^\rightarrow$
f_4	0.46221	0.46221	0.46221	0.46221	0.46221
g_3	0.33332	0.33332	0.33332	0.33332	0.33332
g_4	0.79121	0.79121	0.79121	0.79121	0.79121

Rule Name	Λ_c°	$\Lambda_c^\leftarrow$	$\Lambda_c^\nearrow$	$\Lambda_c^\uparrow$	$\Lambda_c^\nwarrow$
f_4	0.25227	0.46221	0.46221	0.46221	0.46221
g_3	0.22625	0.33332	0.33332	0.33332	0.33332
g_4	0.16811	0.79121	0.79121	0.79121	0.79121

Table 7. Analysis of information cooking.

These information cooking rates are also not fixed, and a margin of error needs to be considered. Tables 8 and 4 show the margin of error of different rules in the model. It is found that the margin of error is very small for all the rules. Hence, during different runs, although the information cooking rate of the suggested model fluctuates a little, a nonmaximal cooking of information is always maintained in every possible direction.

Rule Name	$\Lambda_c^\star$	$\Lambda_c^\succ$	$\Lambda_c^\downarrow$	$\Lambda_c^\swarrow$	$\Lambda_c^\rightarrow$
f_4	0.001834	0.001592	0.001834	0.001834	0.001834
g_3	0.001599	0.001599	0.001599	0.001599	0.001599
g_4	0.001510	0.001510	0.001510	0.001510	0.001510

Rule Name	Λ_c°	$\Lambda_c^\leftarrow$	$\Lambda_c^\nearrow$	$\Lambda_c^\uparrow$	$\Lambda_c^\nwarrow$
f_4	0.001281	0.001834	0.001834	0.001834	0.001834
g_3	0.001332	0.001599	0.001599	0.001599	0.001599
g_4	0.001296	0.001510	0.001510	0.001510	0.001510

Table 8. Margin of error for calculating information cooking of all the rules.

5.1 Experimental Results of SARS 2003 Simulation

Figure 6 displays results from the SARS model simulation. The initial assumption for the simulation is the same as in the previous case. Figure 6(a) shows the initial configuration. The other figures (Figure 6(b) to Figure 6(d)) show the intermediate configurations. Figure 7 shows the graph representation of the simulation, whereas Figure 7(a) shows the growth of infected cells with respect to time. Figure 7(b) shows the growth of recovered cells. Figure 7(c) shows the combined graph, Figure 7(d) shows the total infected cells present for different time periods, and Figure 5 depicts the real-time scenario of China. Here we use the SARS 2003 spreading dataset from www.kaggle.com/datasets/imdevskp/corona-virus-report and use three cases: the wave, recovered cases and confirmed cases. We pre-process the dataset approximately to fit into the graph. The simulation results are then compared with the real-time data to validate the proposed model. Figures 7 and 8 show that the model presents identical circumstances as a real-time scenario. Hence, our proposed model effectively simulates the spreading.

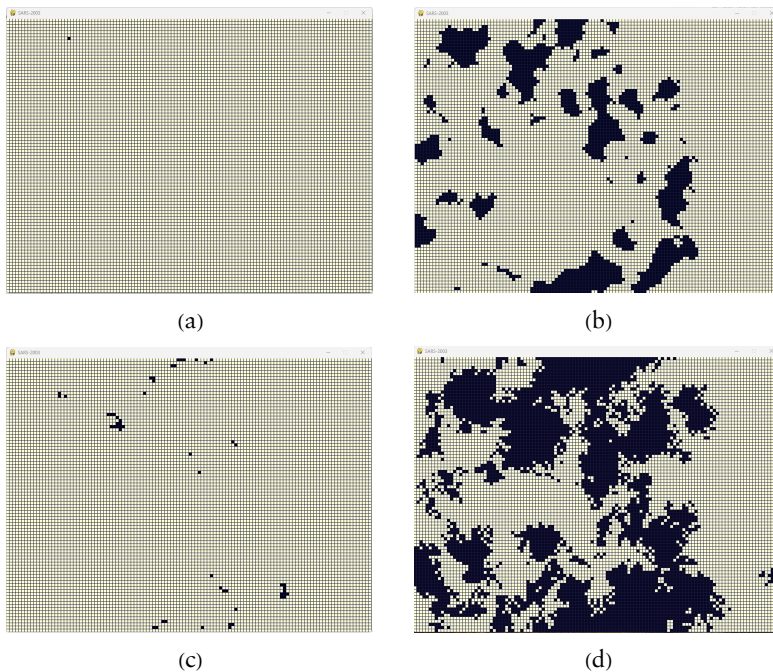


Figure 6. (*continues*)

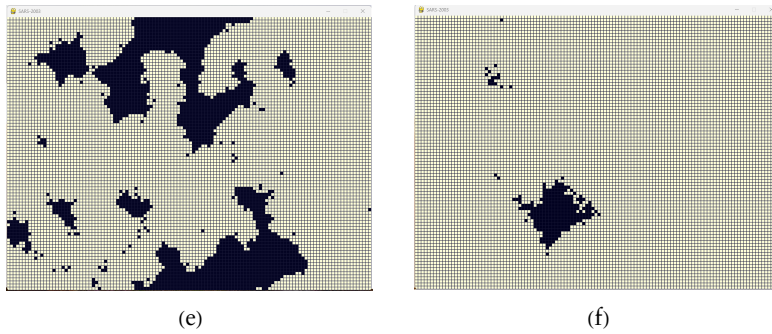


Figure 6. Simulation of SARS 2003: (a) initial configuration; (b) intermediate configuration; (c) configuration with low infection rate; (d) high rate of infection; (e) intermediate configuration; (f) stabilizing the simulator.

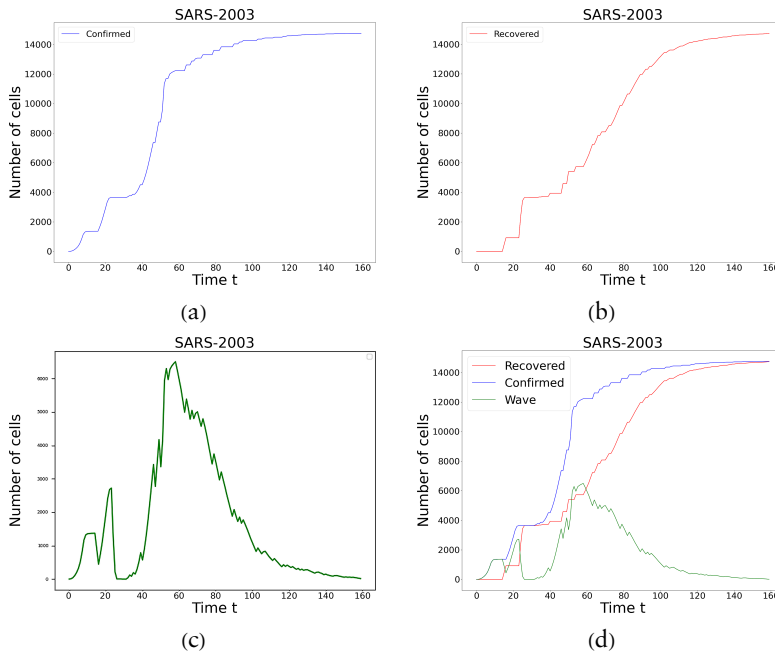


Figure 7. Simulation graphs of SARS 2003; (a) confirmed cases over time; (b) recovered cells over time; (c) the wave trajectory; (d) combination of (a), (b) and (c).

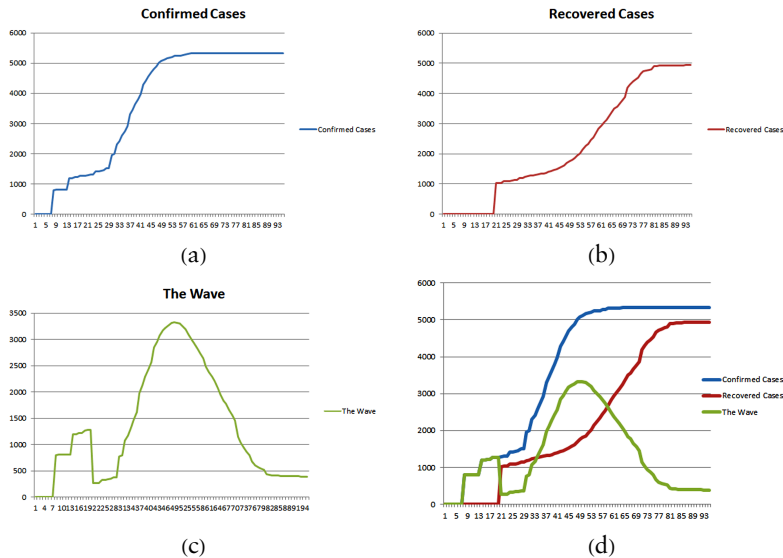


Figure 8. Graphs of real-time data from China (a) confirmed cases over time; (b) recovered cells over time; (c) the wave trajectory; (d) combination of (a), (b) and (c).

6. Discussion

This paper highlights the potential of nonuniform temporally stochastic cellular automata (NTSCAs) to mimic the behavior of any virus that is very infectious and may cause a pandemic. Our model includes several features that can be observed in transmitting highly contagious viruses. In our NTSCAs, each cell updates depending on itself, eight nearest neighbors and a distant neighbor (with some probability p). When a disease starts spreading from a long-distance communication, this distant neighbor becomes essential for managing the problem globally. To efficiently simulate the viral mutation, each temporally stochastic cellular automaton (TSCA) (say, $((f, g)[\tau])|_t$) has been changed throughout time. It depicts the variation of spread in different circumstances.

We have considered two pandemic situations: spread of COVID-19 in India and SARS 2003 in China. Finally, we validate the outcome of the simulation with real-time datasets. The comparison of results has been reported and these are quite converging. These outcomes depict that our model can simulate the spread and behavior of such highly contagious viruses. So this model may be used to mimic a number of different infectious viruses, including influenza, HIV, herpes and H1N1.

Acknowledgments

This paper was started as a project in the Indian Summer School on Cellular Automata 2022. The authors gratefully acknowledge Prof. Sukanta Das and Dr. Souvik Roy for their valuable comments and suggestions. This work is partially supported by Start-up Research Grant (File number SRG/2022/002098), SERB, Department of Science & Technology, Government of India.

References

- [1] WHO: Update 1: Severe Acute Respiratory Syndrome (SARS). (16 March 2003)
www.who.int/emergencies/disease-outbreak-news/item/2003_03_16-en.
- [2] WHO: Summary of Probable SARS Cases with Onset of Illness from 1 November 2002 to 31 July 2003. (21 April 2004)
www.who.int/publications/m/item/summary-of-probable-sars-cases-with-onset-of-illness-from-1-november-2002-to-31-july-2003.
- [3] Council on Foreign Relations. “Major Epidemics of the Modern Era.” (Jan 16, 2025) www.cfr.org/timeline/major-epidemics-modern-era.
- [4] J. Piret and G. Boivin, “Pandemics throughout History,” *Frontiers in Microbiology*, **11**, 2020 631736. doi:10.3389/fmicb.2020.631736.
- [5] C. Beauchemin, J. Samuel and J. Tuszynski, “A Simple Cellular Automaton Model for Influenza A Viral Infections,” *Journal of Theoretical Biology*, **232**(2), 2005 pp. 223–234. doi:10.1016/j.jtbi.2004.08.001.
- [6] P. Podolski and H. S. Nguyen, “Cellular Automata in COVID-19 Prediction,” *Procedia Computer Science*, **192**, 2021 pp. 3370–3379. doi:10.1016/j.procs.2021.09.110.
- [7] A. Cavalcante, L. Faria, M. da Costa Lemos, M. Farias and H. Carvalho, “Modelling the Spread of COVID-19 in the Capital of Brazil Using Numerical Solution and Cellular Automata,” *Computational Biology and Chemistry*, **94**, 2021 107554. doi:10.1016/j.compbiolchem.2021.107554.
- [8] S. H. White, A. M. del Rey and G. R. Sánchez, “Modeling Epidemics Using Cellular Automata,” *Applied Mathematics and Computation*, **186**(1), 2007 pp. 193–202. doi:10.1016/j.amc.2006.06.126.
- [9] B. Cissé, S. El Yacoubi and A. Tridane, “Impact of Neighborhood Structure on Epidemic Spreading by Means of Cellular Automata Approach,” *Procedia Computer Science*, **18**, 2013 pp. 2603–2606. doi:10.1016/j.procs.2013.05.450

- [10] G. Ortigoza, F. Brauer, and I. Neri, “Modelling and Simulating Chikungunya Spread with an Unstructured Triangular Cellular Automata,” *Infectious Disease Modelling*, 5(1), 2020 pp. 197–220. doi:10.1016/j.idm.2019.12.005.
- [11] S. Roy, S. Paul and S. Das, “Temporally Stochastic Cellular Automata: Classes and Dynamics,” *International Journal of Bifurcation and Chaos*, 32(12), 2022 2230029. doi:10.1142/S0218127422300294.
- [12] S. Paul and K. Bhattacharjee, “Modeling Spread of Contagious Disease by Temporally Stochastic Cellular Automata,” in *Proceedings of Second Asian Symposium on Cellular Automata Technology (ASCAT 2023)* (S. Das and G. J. Martinez, eds.), Singapore: Springer Singapore, 2023 pp. 161–175. doi:10.1007/978-981-99-0688-8_13.
- [13] S. Kamilya and S. Das, “A Study of Chaos in Cellular Automata,” *International Journal of Bifurcation and Chaos*, 28(3), 2018 1830008. doi:10.1142/S0218127418300082.
- [14] W. G. Cochran, *Sampling Techniques*, 3rd ed., New York: Wiley, 1977.
- [15] S. Paul and K. Bhattacharjee. “Modeling Spread of Contagious Disease by Temporally Stochastic Cellular Automata.” (Jan 16, 2025) github.com/subrata29/Covid-19-model.